Desarrollo de Software para
Inteligencia Artificial Generativa
API RAG para Selección Inteligente de Vehículo

ARQUITECTURA Y DESARROLLO PARA INTELIGENCIA ARTIFICIAL GENERATIVA

Informe técnico. Caso de aplicación

API RAG para selección de vehículo

Propuesta de servicio agnóstico al consumidor, construido en Python

Autores:
Edward Alejandro Rayo Cortés
Elizabeth Toro Chalarca
Santiago Andrés Cardona Julio

Medellín
2025

Historia

Versión	Fecha	Descripción del cambio
1	20250410	Inicio. Definición de la funcionalidad RAG y de la base de conocimiento. Propuesta de arquitectura y documentación del caso de aplicación. Replanteamiento como API agnóstica al consumidor. Adopción de arquitectura hexagonal y tono propositivo del documento.

CONTENIDO

1. API RAG para selección inteligente de vehículo.... 4
1.1 Presentación de la funcionalidad RAG.... 5
1.2 Esquema explicativo de la funcionalidad RAG.... 5
1.3. Análisis comparativo de funcionalidades candidatas.... 7
2. Elementos de datos en la funcionalidad RAG.... 8
2.1 Definición de la base de conocimiento.... 8
2.2 Definición de entradas.... 9
2.3 Definición de salidas.... 11
3. Propuesta de arquitectura de la solución.... 13
3.1 Adaptadores de entrada.... 14
3.2 Núcleo hexagonal.... 16
3.3 Puertos definidos por el núcleo.... 16
3.4 Adaptadores de salida. Repositorio de conocimiento.... 18
3.5 Adaptadores de salida. Proveedores de LLM.... 19
3.6 Adaptadores de salida. Proveedores de embeddings.... 20
3.7 Pipeline de ingestión.... 20
3.8 Beneficios del diseño hexagonal.... 21
3.9 Consideraciones de despliegue y operación.... 22
4. Asignación de tareas del equipo.... 23
5. Conclusiones del caso de aplicación.... 24
6. Referencias.... 25
Apéndice A. Glosario de siglas y abreviaturas.... 27

1. API RAG para selección inteligente de vehículo

La selección del vehículo de transporte adecuado para cada operación logística en el sector agrícola constituye una decisión que impacta de forma directa los costos operativos, los tiempos de entrega y la calidad del producto entregado. La decisión depende de múltiples variables, entre ellas el tipo de producto, el volumen de la carga, las condiciones climáticas, el estado de las vías, la capacidad disponible de la flota, las restricciones de refrigeración y los costos asociados a cada opción de transporte. Cuando esta decisión se toma de manera manual o con base en reglas estáticas, surgen ineficiencias como la subutilización de la capacidad vehicular, la asignación inadecuada del tipo de transporte y los sobrecostos logísticos derivados.

Como respuesta a este problema, se propone la implementación de una API RAG denominada API de Selección Inteligente de Vehículos. Esta API se construirá como servicio autónomo en Python, con una interfaz HTTP agnóstica al consumidor. De esta manera, cualquier sistema cliente autorizado enviará una solicitud con los datos del pedido y la flota disponible, y la API retornará una recomendación fundamentada del vehículo óptimo. La API integrará la capacidad de razonamiento de un modelo de lenguaje grande (LLM) con una base de conocimiento documental especializada en logística agrícola colombiana.

La decisión de construir la funcionalidad como API agnóstica responde a tres criterios. En primer lugar, la independencia tecnológica: cualquier sistema consumidor, sin importar su stack (Java, .NET, Node.js, otro servicio en Python), se integra mediante HTTP estándar. En segundo lugar, la cohesión del dominio: el servicio encapsula toda la lógica de inferencia con modelos de lenguaje, la gestión de embeddings y el acceso a la base de conocimiento, sin mezclarse con lógica de otros dominios. En tercer lugar, el escalamiento autónomo: el perfil de consumo de recursos de un servicio RAG (memoria para embeddings, latencia por llamadas al LLM) difiere del perfil de los sistemas transaccionales, por lo cual conviene escalarlo de forma independiente.

El resultado esperado es que el coordinador logístico reciba, para cada pedido, una recomendación argumentada que indique el vehículo más adecuado de la flota disponible, junto con la justificación de la selección, las alternativas consideradas y las alertas relevantes sobre condiciones de la ruta o del producto. A nivel de negocio, esto se traduce en tres beneficios concretos. Primero, la reducción de costos logísticos, al asignar vehículos con la capacidad ajustada a la carga y evitar el uso de transporte sobredimensionado. Segundo, la disminución de pérdidas por deterioro, al considerar las condiciones de refrigeración, ventilación y tiempo máximo de tránsito según el tipo de producto agrícola transportado. Tercero, la adaptabilidad ante cambios del entorno: a diferencia de un sistema de reglas fijas, el servicio incorpora nuevo conocimiento mediante la actualización de su base documental, sin necesidad de modificar el código ni desplegar la aplicación, lo que lo mantiene vigente frente a cambios normativos, nuevas rutas o modificaciones en la flota.

1.1 Presentación de la funcionalidad RAG

La API funcionará como un asistente de decisión accesible por HTTP. Ante cada solicitud entrante, analizará múltiples dimensiones del problema logístico y producirá una recomendación argumentada. El servicio no se limitará a una asignación por reglas, sino que empleará la capacidad de razonamiento del LLM para considerar interacciones complejas entre variables. En particular, contemplará la relación entre el tipo de producto perecedero, las condiciones climáticas de la ruta y la disponibilidad de vehículos con cadena de frío.

La justificación de esta funcionalidad se sustenta en tres factores. En primer lugar, la reducción de costos logísticos: al seleccionar el vehículo con la capacidad ajustada a la carga y el tipo de transporte más eficiente para la ruta, se evitarán desperdicios de capacidad y sobrecostos por vehículos sobredimensionados. En segundo lugar, la preservación de la calidad del producto: al considerar las condiciones de refrigeración, ventilación y tiempo máximo de tránsito según el tipo de producto, se reducirán las pérdidas por deterioro. En tercer lugar, la adaptabilidad: a diferencia de un sistema de reglas fijas, el RAG incorporará nuevos conocimientos mediante la actualización de la base documental, sin modificaciones en el código ni despliegues nuevos del servicio.

En el contexto colombiano, donde la geografía diversa presenta retos importantes para el transporte agrícola, como las vías terciarias en mal estado, las variaciones altitudinales que afectan la cadena de frío y las restricciones de acceso por tamaño de vehículo en zonas rurales, esta funcionalidad resulta de particular valor al integrar conocimiento especializado sobre las condiciones locales de transporte.

1.2 Esquema explicativo de la funcionalidad RAG

El esquema de funcionamiento de la API se articula en cuatro fases operativas que reflejan el flujo desde la recepción de la solicitud hasta la entrega de la respuesta al consumidor. En la Figura 1 se presenta dicho flujo.

Fase 1. Recepción de la solicitud. El consumidor envía una petición HTTP POST al endpoint del servicio con un cuerpo JSON que contiene los datos del pedido (cliente de destino, productos solicitados con sus cantidades, canal de comercialización, prioridad) y la lista de vehículos disponibles. La API valida el payload con esquemas Pydantic y rechaza solicitudes mal formadas antes de invocar la lógica de dominio.

Fase 2. Recuperación de contexto. El servicio construye una consulta semántica a partir de los datos del pedido, la convierte en un vector de embedding y consulta la base de conocimiento a través del puerto KnowledgeRepository. Dicho puerto retorna los fragmentos documentales más relevantes por similitud, enriquecidos con entidades relacionadas recuperadas de la base de grafos.

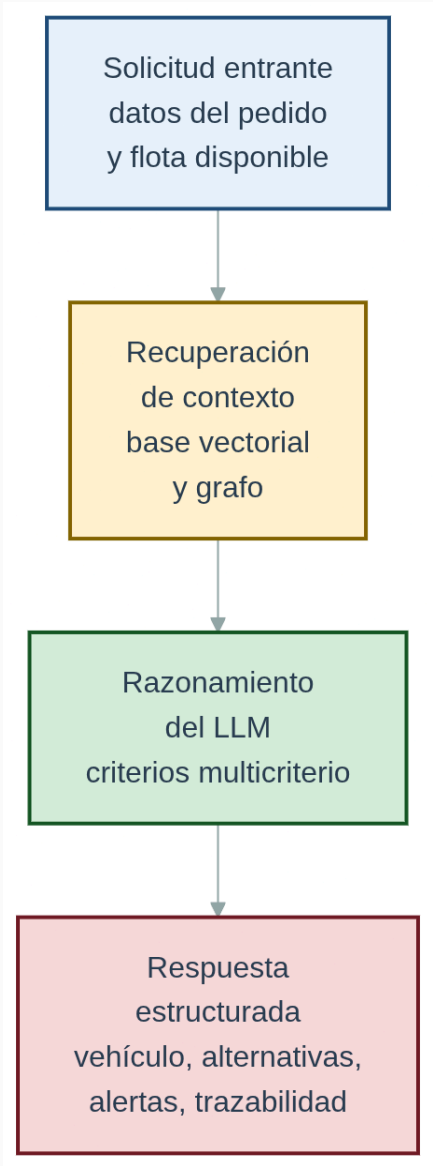
Fase 3. Razonamiento del LLM. El servicio ensambla el prompt final combinando las instrucciones del sistema, el contexto recuperado y los datos de la solicitud. Posteriormente, envía el prompt al proveedor del LLM. El modelo aplica criterios de optimización en términos de costo, tiempo y calidad, y genera una recomendación estructurada que incluye el tipo de

vehículo sugerido, la justificación de la selección, las alternativas consideradas y las alertas relevantes.

Fase 4. Respuesta al consumidor. La API parsea la respuesta del LLM con un ResponseParser que transforma el texto en un objeto JSON estructurado. La respuesta se retorna al consumidor con el código HTTP 200 y un identificador de trazabilidad (trace_id) que vincula la inferencia con los fragmentos documentales consultados y la respuesta del LLM.

Como se observa en la Figura 1, el esquema representa las cuatro fases descritas y las relaciones entre los elementos principales del flujo.

Figura 1. Esquema explicativo de la funcionalidad RAG.



1.3. Análisis comparativo de funcionalidades candidatas

Antes de seleccionar la funcionalidad de selección inteligente de vehículo, el equipo evaluó cinco alternativas de funcionalidad RAG aplicables al dominio de distribución agrícola. Con el fin de formalizar la decisión, se construyó una matriz de calificación con cinco criterios ponderados. La calificación se realizó en una escala de 1 a 5. En la Tabla 1 se presentan los criterios con su peso y su justificación.

Tabla 1. Criterios de evaluación y su peso.

Criterio	Peso	Justificación del criterio
Valor para el negocio	25%	Mide el impacto directo de la funcionalidad en reducción de costos, incremento de ingresos o retención de clientes.
Adecuación al enfoque RAG	25%	Evalúa qué tan bien encaja el problema con Retrieval-Augmented Generation frente a alternativas como ML (regresión, clasificación), optimización combinatoria o reglas de negocio.
Facilidad de adquisición de datos	20%	Mide la disponibilidad y accesibilidad de los documentos y datos necesarios para construir la base de conocimiento y las entradas de la API.
Complejidad de implementación	15%	Evalúa el esfuerzo técnico para construir un MVP. Escala invertida: 5 indica baja complejidad (más favorable).
Explicabilidad de la salida	15%	Mide la importancia de que el sistema entregue una justificación en lenguaje natural que el usuario final pueda auditar.

En la Tabla 2 se presentan los resultados de la evaluación. Cada celda contiene la calificación (1 a 5) y la justificación correspondiente.

Tabla 2. Matriz de evaluación de funcionalidades RAG candidatas.

Funcionalidad	Valor negocio (25%)	Adecuación RAG (25%)	Datos (20%)	Complejidad (15%)	Explicabilidad (15%)
Selección de vehículo (seleccionada)	**5** Impacta costo logístico, merma y puntualidad.	**4** El razonamiento sobre documentos heterogéneos se alinea con RAG. Un ML requiere datos históricos no disponibles.	**4** Fichas técnicas y normativa de acceso público o institucional.	**4** Arquitectura hexagonal con FastAPI permite construir un MVP con esfuerzo acotado.	**5** El coordinador requiere entender por qué se recomienda un vehículo. La explicación es parte del valor.
Empaque óptimo	**4** Reduce costo de material y merma. Impacto medible pero menor.	**2** Problema de clasificación tabular. Un ML con datos históricos sería más preciso.	**3** Fichas de producto existen. Datos de empaque por operación no se registran.	**4** Baja complejidad. Variables de decisión reducidas.	**3** Explicación útil pero no es el eje del valor.
Separación FEFO	**5** Impacta merma por vencimiento, entre 5% y 8% del inventario mensual.	**1** Problema de optimización combinatoria. Se resuelve con programación lineal. RAG no aporta valor diferencial.	**4** Datos de lotes y fechas de elaboración existen en el sistema transaccional.	**2** Alta complejidad. Requiere modelado matemático y validación formal.	**2** Salida numérica. Explicación en lenguaje natural es complementaria.

Recomendador de canal	**5** Impacta margen bruto por lote. Mejora del 2% en margen anual.	**3** RAG aporta valor con datos escasos. Con datos ricos, ML lo supera.	**3** SIPSA [20] es público. Datos de clientes con precios pactados están fragmentados.	**3** Complejidad media. Integración de fuentes externas e internas.	**4** Coordinador comercial valora la explicación del trade-off.
Incidencias en ruta	**4** Protege valor ya generado. Impacto alto pero infrecuente.	**5** Encaje natural con RAG. Eventos raros sin volumen de datos para ML.	**2** Protocolos internos no están digitalizados. Requiere esfuerzo previo.	**3** Complejidad media. Diversidad de escenarios exige prompt engineering sofisticado.	**5** El plan de acción en lenguaje natural es el entregable.

En la Tabla 3 se presenta el puntaje ponderado total de cada funcionalidad.

Tabla 3. Puntaje ponderado total por funcionalidad.

Funcionalidad	Puntaje ponderado	Resultado
Selección de vehículo	**4,40**	**Seleccionada**
Incidencias en ruta	3,85	Segunda opción
Recomendador de canal	3,65	Tercera opción
Empaque óptimo	3,15	Descartada
Separación FEFO	2,90	Descartada

Como se observa en la Tabla 3, la funcionalidad de selección de vehículo obtiene el puntaje más alto (4,40 sobre 5,00). Este resultado se explica por su combinación de alto valor para el negocio, buena adecuación al enfoque RAG, datos accesibles desde fuentes públicas e institucionales, complejidad acotada para un MVP y una salida cuyo valor reside en la explicación al coordinador logístico.

2. Elementos de datos en la funcionalidad RAG

Esta sección describe los tres componentes centrales del flujo de datos del servicio. En primer lugar, la base de conocimiento, que alimentará la recuperación de información. En segundo lugar, las entradas del contrato de API. En tercer lugar, las salidas que la API retornará como resultado del razonamiento.

2.1 Definición de la base de conocimiento

La base de conocimiento constituirá el repositorio documental especializado que el servicio consultará en cada inferencia. Se organizará en cinco categorías documentales. En la Tabla 4 se presenta el contenido de cada categoría, junto con su formato y su fuente de origen.

Tabla 4. Categorías documentales de la base de conocimiento.

Categoría	Contenido	Formato y fuente
Fichas técnicas de productos	Requisitos de temperatura, humedad, ventilación, fragilidad, vida útil y	Documentos en PDF y Markdown. Fuentes: ICA [15],

	condiciones especiales de transporte para cada tipo de producto agrícola, entre ellos frutas, verduras, granos y flores.	Corpoica/AGROSAVIA [16] y manuales de buenas prácticas agrícolas.
Catálogo de flota vehicular	Especificaciones de cada vehículo: capacidad en kilogramos y metros cúbicos, tipo de carrocería (refrigerada, isotérmica o abierta), rango de temperatura, autonomía, restricciones de acceso por dimensiones y costo por kilómetro.	JSON estructurado. El mantenimiento estará a cargo del equipo encargado de la flota.
Condiciones de rutas y vías	Estado de vías terciarias, restricciones por peso o tamaño, tiempos estimados de recorrido, peajes y zonas con riesgo de deslizamiento según la temporada.	Datos de INVIAS [17] y reportes de servicios de mapas.
Tarifas y costos de transporte	Tablas de tarifas por tipo de vehículo, distancia, zona geográfica y tipo de carga. Costos asociados al combustible, los peajes, los seguros y los viáticos.	Hojas de cálculo y documentos del área financiera. Actualización mensual.
Normativa de transporte agrícola	Regulaciones del Ministerio de Transporte [19] sobre transporte de alimentos, normas del INVIMA [18] para la cadena de frío y restricciones de circulación por horario y tipo de vehículo.	Documentos normativos en PDF. Resoluciones del MinTransporte e INVIMA.

Como se observa en la Tabla 4, los documentos se procesarán mediante un pipeline de ingestión ejecutado en Python. La primera etapa corresponderá a la extracción de texto con las librerías pypdf y pdfplumber, complementadas con Tesseract OCR para documentos escaneados. La segunda etapa consistirá en la segmentación en chunks de 500 a 1000 tokens mediante el Recursive CharacterText Splitter de LangChain [12], con un solapamiento del 10%. La tercera etapa comprenderá la generación de embeddings. La cuarta etapa corresponderá a la persistencia combinada en dos almacenes: una base vectorial (ChromaDB) para búsquedas por similitud semántica y una base de grafos (Neo4j) donde se modelarán las entidades del dominio y sus relaciones. Cada chunk conservará metadatos de origen (categoría, fecha de actualización y fuente) para facilitar el filtrado durante la recuperación.

2.2 Definición de entradas

Las entradas del servicio se definen como un contrato de API en formato JSON. Cualquier sistema consumidor deberá construir un payload que cumpla con los campos y las restricciones documentadas. En la Tabla 5 se detalla cada campo del payload de entrada.

Tabla 5. Campos del payload de entrada a la API.

Campo	Descripción	Condiciones y reglas de validación

pedido	Objeto con los datos básicos del pedido: identificador, fecha de entrega y prioridad.	Identificador alfanumérico no vacío. Fecha en formato ISO 8601. Prioridad dentro del conjunto (alta, media, baja).
productos	Arreglo con los productos incluidos en el pedido, con nombre, cantidad y unidad de medida.	Arreglo no vacío. Cada producto debe tener cantidad mayor que cero y unidad dentro del conjunto permitido (kg, ton, unidades).
cliente	Datos del destinatario: nombre, dirección, latitud y longitud.	La dirección debe estar geocodificada con coordenadas válidas en el rango geográfico de Colombia.
canal	Canal comercial por el cual se mueve el pedido.	Valor dentro del conjunto (mayorista, minorista, exportacion, institucional).
flota_disponible	Arreglo con los vehículos elegibles para la asignación en la fecha requerida.	Arreglo no vacío. Cada elemento con id único, tipo (TERRESTRE, AEREO, ACUATICO), capacidad en kg y bandera de refrigeración.

El contrato de la API se formalizará mediante esquemas Pydantic, lo cual permitirá validar el payload al recibirlo y retornar errores claros con código HTTP 422 cuando los datos no cumplan las condiciones establecidas. En el Fragmento 1 se presenta un ejemplo del payload.

Fragmento 1. Ejemplo de payload JSON enviado a la API.

```
POST /api/v1/vehicle-recommendation HTTP/1.1
Host: rag-service.example.com
Authorization: Bearer <service-token>
Content-Type: application/json

{
  "pedido": {
    "identificador": "PED-2025-0042",
    "fecha_entrega": "2025-04-20",
    "prioridad": "alta"
  },
  "productos": [
    {"nombre": "Aguacate Hass", "cantidad": 1200, "unidad": "kg"},
    {"nombre": "Plátano hartón", "cantidad": 800, "unidad": "kg"}
  ],
  "cliente": {
    "nombre": "Distribuidora Andina S.A.",
    "direccion": "Calle 80 # 45-12, Bogotá",
    "latitud": 4.6782,
    "longitud": -74.0584
  },
  "canal": "mayorista",
  "flota_disponible": [
    {"id": "VEH-015", "tipo": "TERRESTRE", "capacidad_kg": 3500, "refrigerado":
true},
    {"id": "VEH-022", "tipo": "TERRESTRE", "capacidad_kg": 2000, "refrigerado":
false}
  ]
}
```

2.3 Definición de salidas

Las salidas del servicio se estructurarán en un objeto JSON que el consumidor procesará según su propia lógica. En la Tabla 6 se describen los campos de la respuesta. En el Fragmento 2 se presenta un ejemplo.

Tabla 6. Campos de la respuesta de la API.

Campo	Descripción	Formato
vehiculo_recomendado	Identificador del vehículo de la flota que mejor se ajusta a las condiciones del pedido, junto con el tipo de transporte y la matrícula.	Objeto JSON con id, tipo y matrícula.
justificacion	Explicación en lenguaje natural del razonamiento que sustenta la selección. Describe por qué el vehículo resulta el más adecuado al considerar la capacidad, el tipo de producto, el costo y el tiempo estimado.	Texto estructurado, con un máximo de 500 palabras.
alternativas	Hasta dos vehículos alternativos con una breve explicación de sus ventajas y desventajas respecto al vehículo principal recomendado.	Arreglo JSON con un máximo de dos alternativas.
alertas	Advertencias sobre condiciones que afectan o podrían afectar el transporte: vías en mal estado, pronóstico de lluvias, producto cercano a vencimiento y capacidad vehicular ajustada.	Arreglo de alertas con nivel de severidad (alta, media, baja).
costo_estimado_cop	Estimación del costo total del transporte, desglosado en combustible, peajes, viáticos del conductor, seguro de la carga y margen de imprevistos.	Valor numérico en COP, con desglose por concepto.
tiempo_estimado_min	Tiempo estimado de recorrido desde el centro de origen hasta el destino del cliente, considerando las condiciones reales de la ruta.	Entero en minutos.
trace_id	Identificador único de la inferencia, útil para trazabilidad y auditoría. Vinculará la recomendación con los fragmentos documentales consultados y la respuesta del LLM.	UUID v4.

Fragmento 2. Ejemplo de respuesta JSON retornada por la API.

```
HTTP/1.1 200 OK
Content-Type: application/json

{
  "trace_id": "7a3f2c1e-8b4d-4e5a-9f6c-2d1b8e0c3a7f",
  "vehiculo_recomendado": {
    "id": "VEH-015",
    "tipo": "TERRESTRE",
    "matricula": "ABC123"
  },
  "justificacion": "El vehículo VEH-015 se selecciona por contar con...",
  "alternativas": [
```

```
    {"id": "VEH-022", "motivo": "Menor costo pero sin refrigeración"}
  ],
  "alertas": [
    {"nivel": "media", "mensaje": "Pronóstico de lluvia en ruta"}
  ],
  "costo_estimado_cop": 485000,
  "tiempo_estimado_min": 420
}
```

La API también definirá códigos HTTP estándar para representar los estados de error. El código 401 se retornará ante ausencia o invalidez del token de autenticación. El código 422 se retornará cuando el payload no cumpla con el esquema definido. El código 500 se retornará ante fallos internos. Cada respuesta de error incluirá un trace_id para facilitar la depuración.

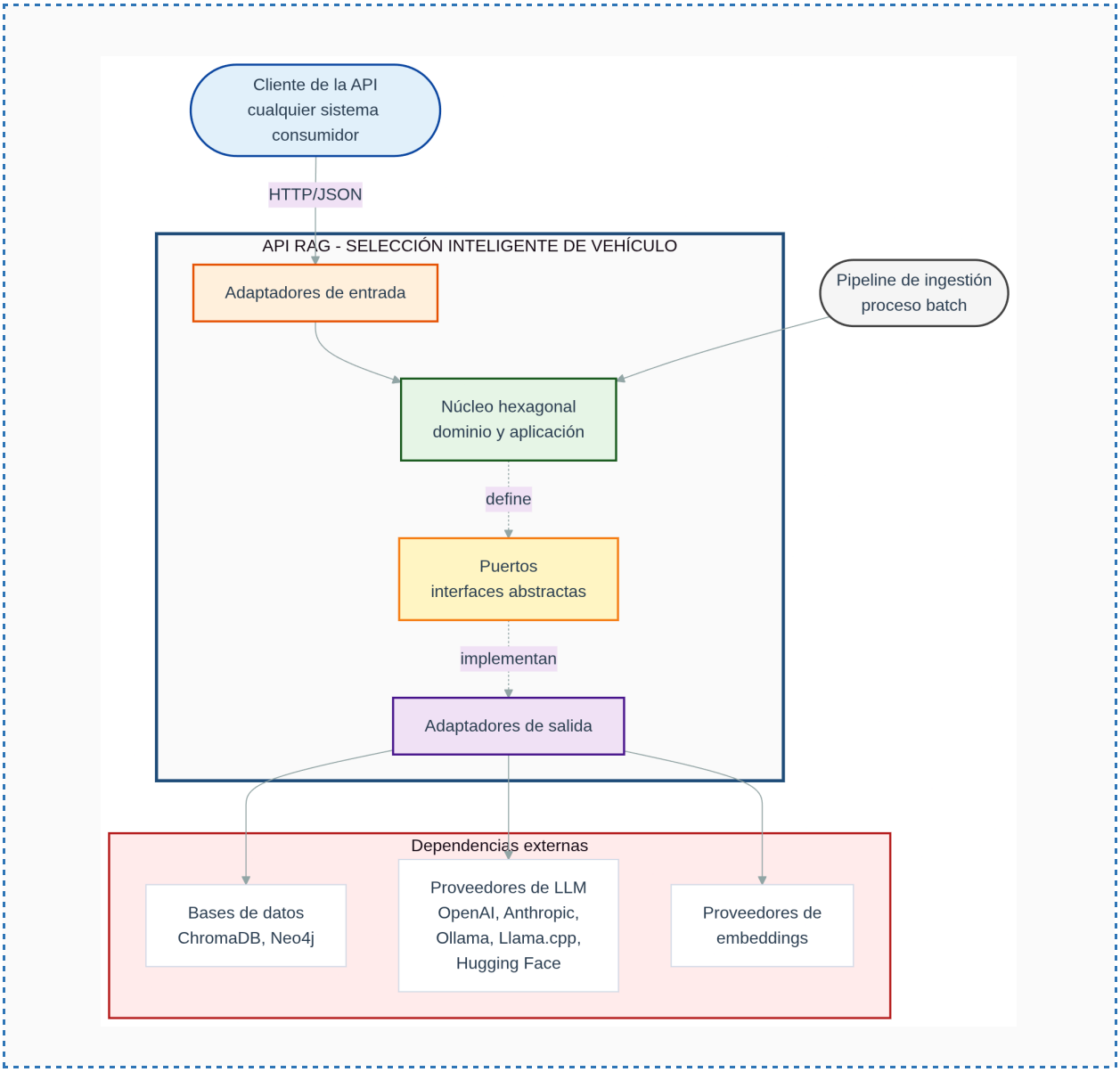
3. Propuesta de arquitectura de la solución

La arquitectura del servicio adoptará el patrón hexagonal, también conocido como ports and adapters, propuesto por Alistair Cockburn [5]. Este patrón separa el núcleo de dominio de la infraestructura externa mediante interfaces (ports) que el núcleo define y que los componentes externos (adapters) implementan. Esta elección arquitectónica obedece a dos necesidades del dominio RAG. La primera es el intercambio de proveedores del modelo de lenguaje sin afectar la lógica del servicio. La segunda es la flexibilidad en el repositorio de conocimiento, que combinará una base vectorial para búsqueda semántica y una base de grafos para el recorrido de relaciones entre entidades del dominio agrícola.

Un criterio de diseño adicional es la delimitación estricta de la responsabilidad del servicio. La API se encargará de todo lo relacionado con el proceso RAG: recuperación de contexto documental, razonamiento con el LLM y generación de la recomendación estructurada. Los datos de ruta, tales como la distancia entre origen y destino, el tiempo estimado de recorrido y las condiciones de la vía, se recibirán como campos de entrada del payload. De esta manera, el cálculo de distancias y tiempos de trayectoria quedará bajo responsabilidad del sistema consumidor, y la API mantendrá su cohesión alrededor del dominio de inferencia.

En la Figura 2 se presenta la vista general del servicio con la relación entre sus capas principales. El servicio se compone de adaptadores de entrada, un núcleo hexagonal, puertos como interfaces abstractas, adaptadores de salida y un pipeline de ingestión independiente.

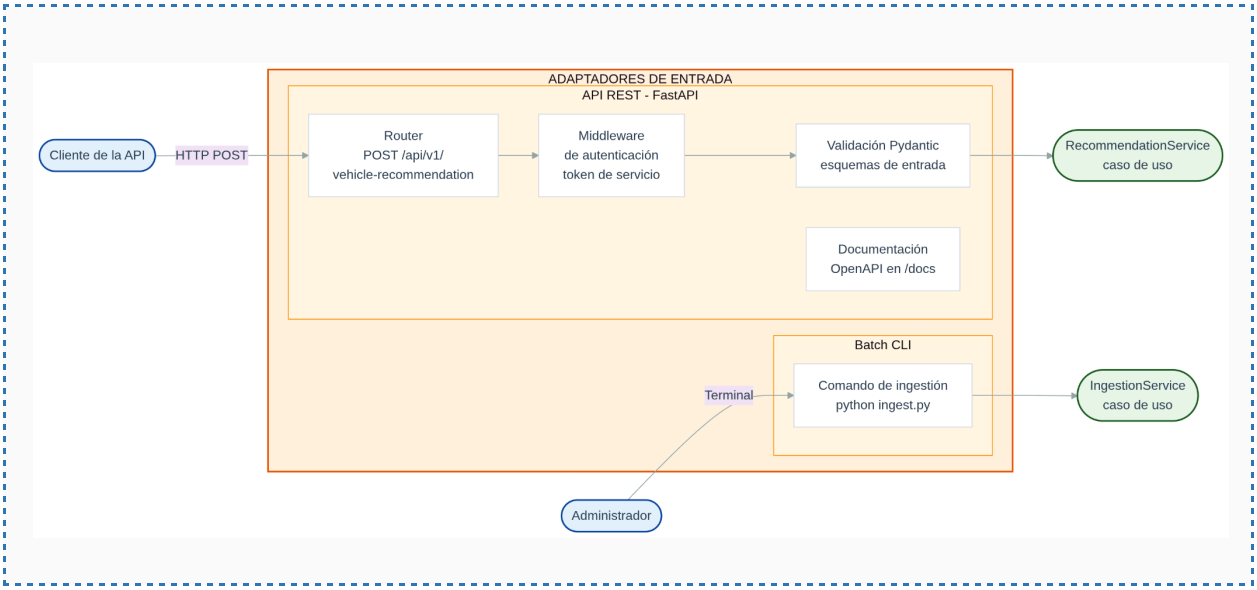
Figura 2. Vista general de la arquitectura hexagonal.



3.1 Adaptadores de entrada

Los adaptadores de entrada traducirán los mecanismos de invocación externos en llamadas a los casos de uso del núcleo. Se implementarán dos adaptadores. En la Figura 3 se presenta la estructura de esta capa.

Figura 3. Capa de adaptadores de entrada.



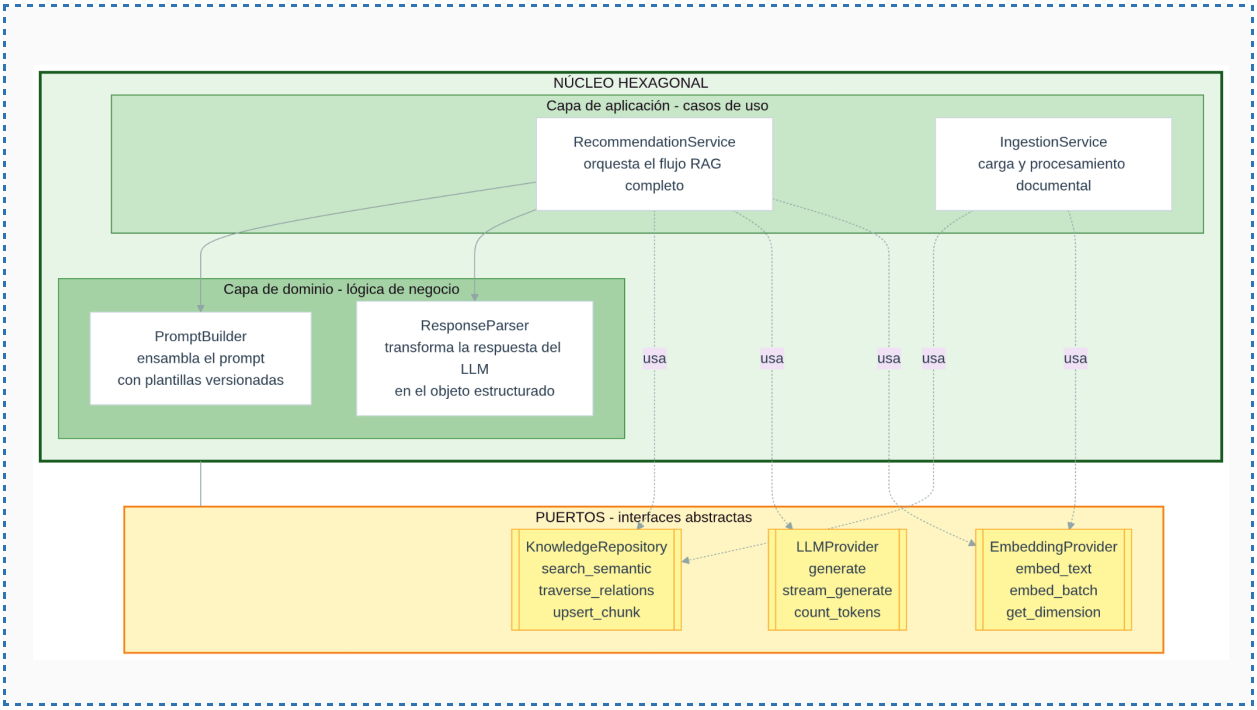
API REST (FastAPI). Expondrá el endpoint POST /api/v1/vehicle-recommendation. Los modelos Pydantic validarán automáticamente el payload entrante y generarán la documentación OpenAPI en /docs. Un middleware de autenticación verificará el token de servicio antes de invocar al RecommendationService.

Batch CLI. Adaptador de línea de comandos que invocará al IngestionService. Se ejecutará bajo demanda o mediante tareas programadas cuando se actualicen los documentos fuente. La separación respecto a la API garantizará que la ingestión no compita por los recursos del endpoint de inferencia.

3.2 Núcleo hexagonal

El núcleo concentrará la lógica de aplicación y de dominio, sin dependencias directas hacia frameworks, bases de datos ni proveedores de LLM. En la Figura 4 se presenta la estructura interna del núcleo y los puertos que define.

Figura 4. Núcleo hexagonal con puertos.



RecommendationService. Caso de uso principal. Orquestará el flujo RAG completo: construirá la consulta semántica, invocará al puerto KnowledgeRepository, ensamblará el prompt con el PromptBuilder, invocará al LLMProvider y finalmente el ResponseParser transformará la respuesta en el objeto de salida.

IngestionService. Caso de uso de carga documental. Ejecutará la extracción, segmentación y vectorización, y registrará los fragmentos a través del puerto KnowledgeRepository.

PromptBuilder. Concentrará la lógica de construcción del prompt, con plantillas versionadas y separación entre instrucciones del sistema, contexto recuperado y consulta del usuario.

ResponseParser. Transformará la respuesta del LLM en un objeto estructurado que cumpla con el contrato de salida, con validaciones defensivas ante respuestas mal formadas.

3.3 Puertos definidos por el núcleo

Los puertos se definirán como interfaces abstractas (Protocol o ABC de Python) que el núcleo consumirá sin conocer su implementación concreta. En la Tabla 7 se describen los tres puertos de la arquitectura.

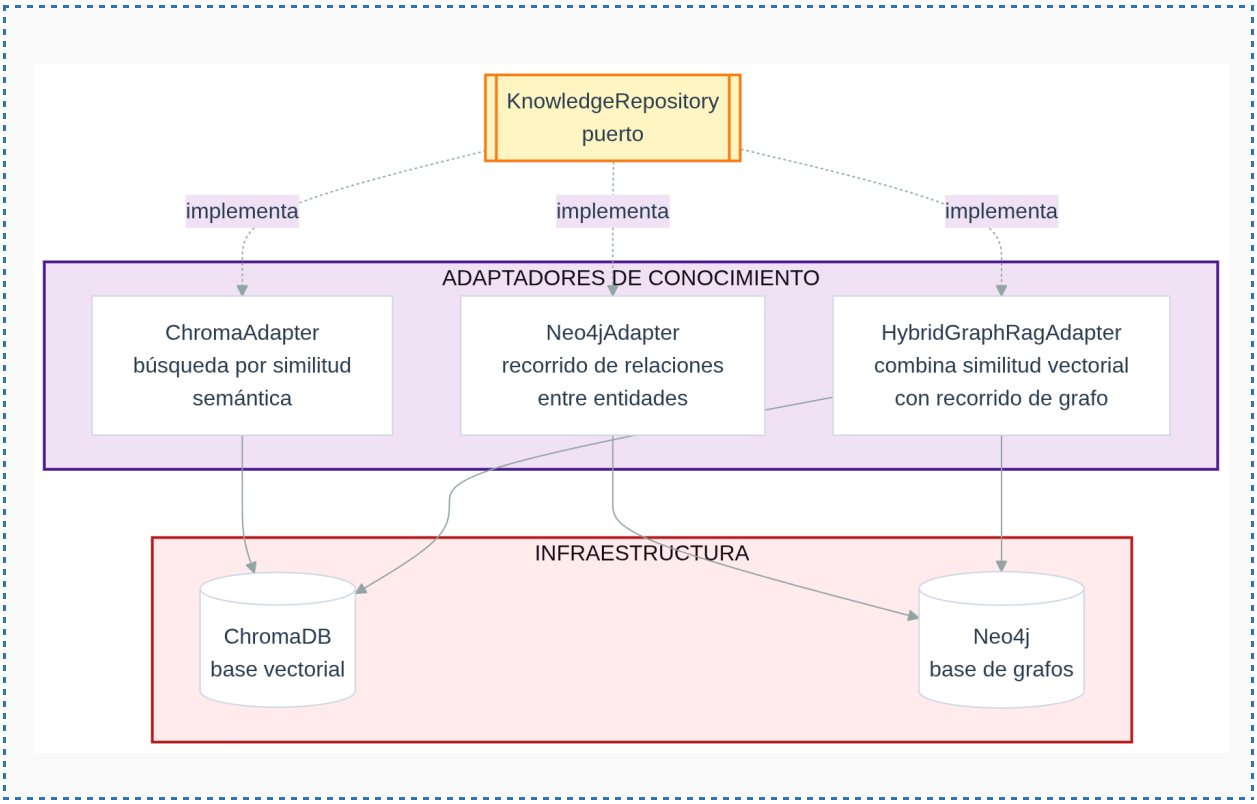
Tabla 7. Puertos del núcleo hexagonal.

Puerto	Responsabilidad	Operaciones principales
KnowledgeRepository	Abstrae el acceso a la base de conocimiento. Oculta al núcleo si la fuente es vectorial, grafo o combinación de ambas.	search_semantic, traverse_relations, upsert_chunk, register_entity_relation, list_by_category.
LLMProvider	Abstrae la invocación al modelo de lenguaje. Desacopla al núcleo del proveedor específico y del protocolo de cada API.	generate, stream_generate, count_tokens.
EmbeddingProvider	Abstrae la vectorización de texto. Permite elegir entre modelos remotos y locales según el costo y la latencia requerida.	embed_text, embed_batch, get_dimension.

3.4 Adaptadores de salida. Repositorio de conocimiento

El puerto KnowledgeRepository contará con tres implementaciones. En la Figura 5 se presenta la relación entre el puerto y sus adaptadores.

Figura 5. Adaptadores del repositorio de conocimiento.



ChromaAdapter. Gestionará la base vectorial ChromaDB y resolverá las búsquedas por similitud semántica. Convertirá la consulta en un vector de embedding, ejecutará una búsqueda de los k fragmentos más similares y retornará los fragmentos con sus metadatos.

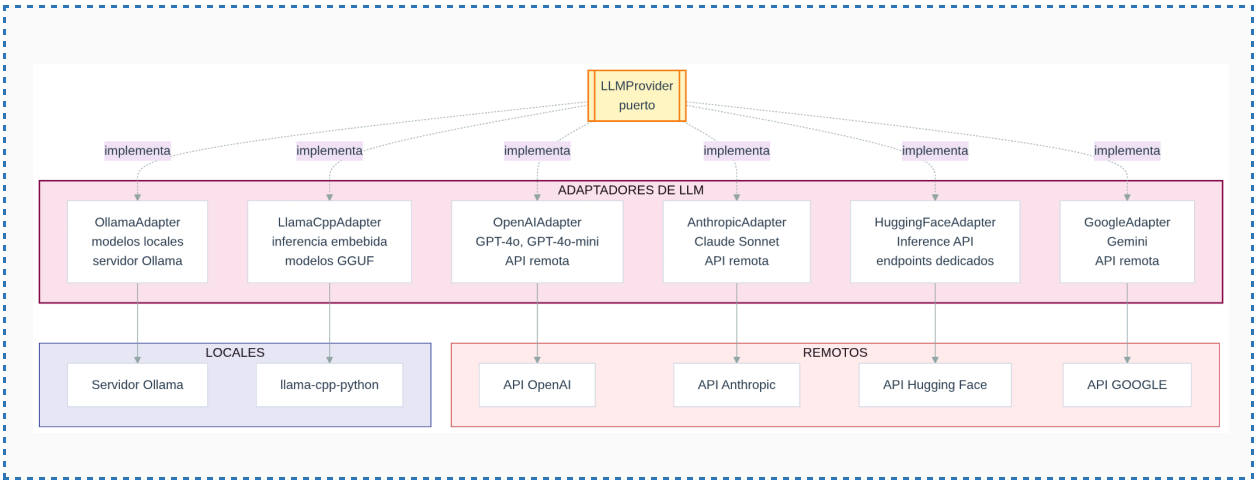
Neo4jAdapter. Gestionará la base de grafos Neo4j, donde se modelarán las entidades del dominio agrícola (productos, vehículos, rutas, normativas) y sus relaciones. Habilitará consultas por recorrido, como por ejemplo los vehículos compatibles con un producto específico según la normativa vigente.

HybridGraphRagAdapter. Combinará ambas fuentes siguiendo el enfoque Graph RAG [21]. Recuperará fragmentos por similitud vectorial y los enriquecerá con entidades relacionadas del grafo. La selección del adaptador activo se configurará mediante variable de entorno, sin modificar el código del núcleo.

3.5 Adaptadores de salida. Proveedores de LLM

El puerto LLMProvider contará con seis implementaciones que cubrirán los escenarios de despliegue comunes. En la Figura 6 se presenta la relación entre el puerto y sus adaptadores.

Figura 6. Adaptadores de proveedores de LLM.



OpenAIAdapter. Consumirá la API de OpenAI para modelos GPT-4o y GPT-4o-mini [4]. Se orientará a escenarios de producción donde se privilegia la calidad del razonamiento.

AnthropicAdapter. Consumirá la API de Anthropic para modelos Claude Sonnet. Ofrecerá una alternativa comercial con características de razonamiento complementarias.

GoogleAdapter. Consumirá la API de Gemini de Google para los modelos de la familia Gemini 1.5. Ofrecerá una opción con tier gratuito y ventana de contexto amplia, adecuada para escenarios donde la base de conocimiento contenga documentos extensos que se procesan directamente como entrada multimodal.

OllamaAdapter. Invocará un servidor Ollama local. Adecuado para ambientes de desarrollo y despliegues con restricciones de confidencialidad.

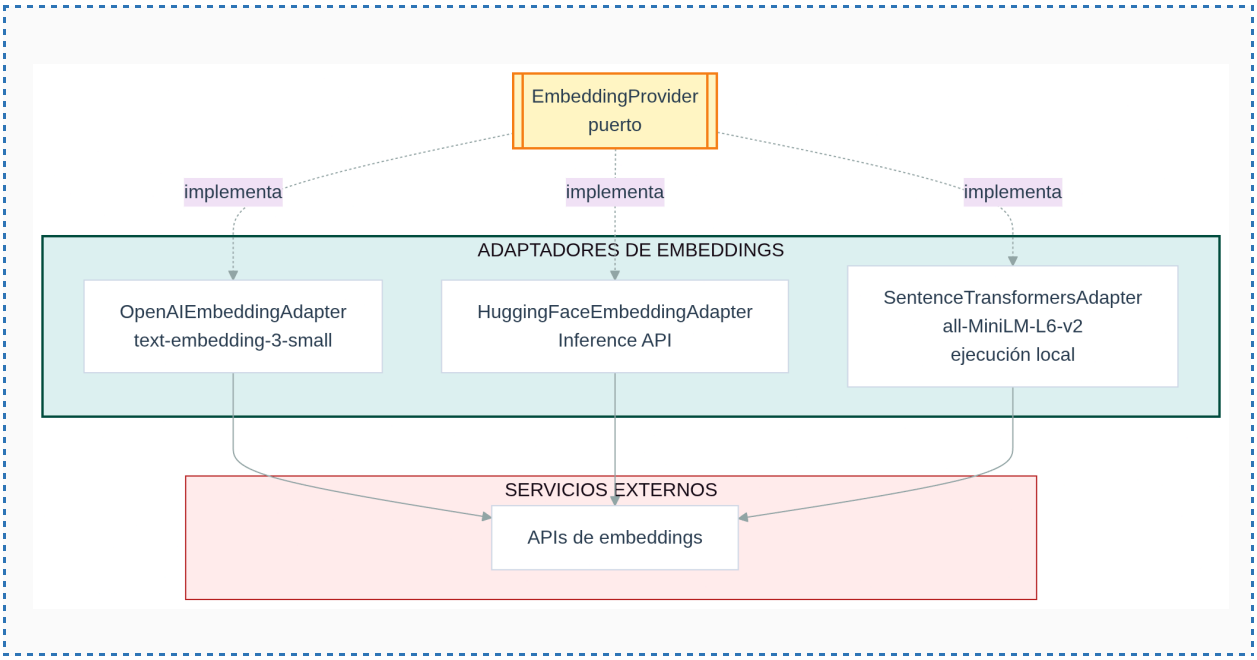
LlamaCppAdapter. Embeberá la inferencia mediante llama-cpp-python con modelos en formato GGUF. Se orientará a entornos con cómputo propio.

HuggingFaceAdapter. Consumirá la Inference API de Hugging Face, con opción de endpoints dedicados. La elección del adaptador se configurará por variable de entorno.

3.6 Adaptadores de salida. Proveedores de embeddings

El puerto EmbeddingProvider abstraerá la vectorización de texto. En la Figura 7 se presenta la relación entre el puerto y sus adaptadores.

Figura 7. Adaptadores de proveedores de embeddings.



OpenAIEmbeddingAdapter. Invocará el modelo text-embedding-3-small de OpenAI.

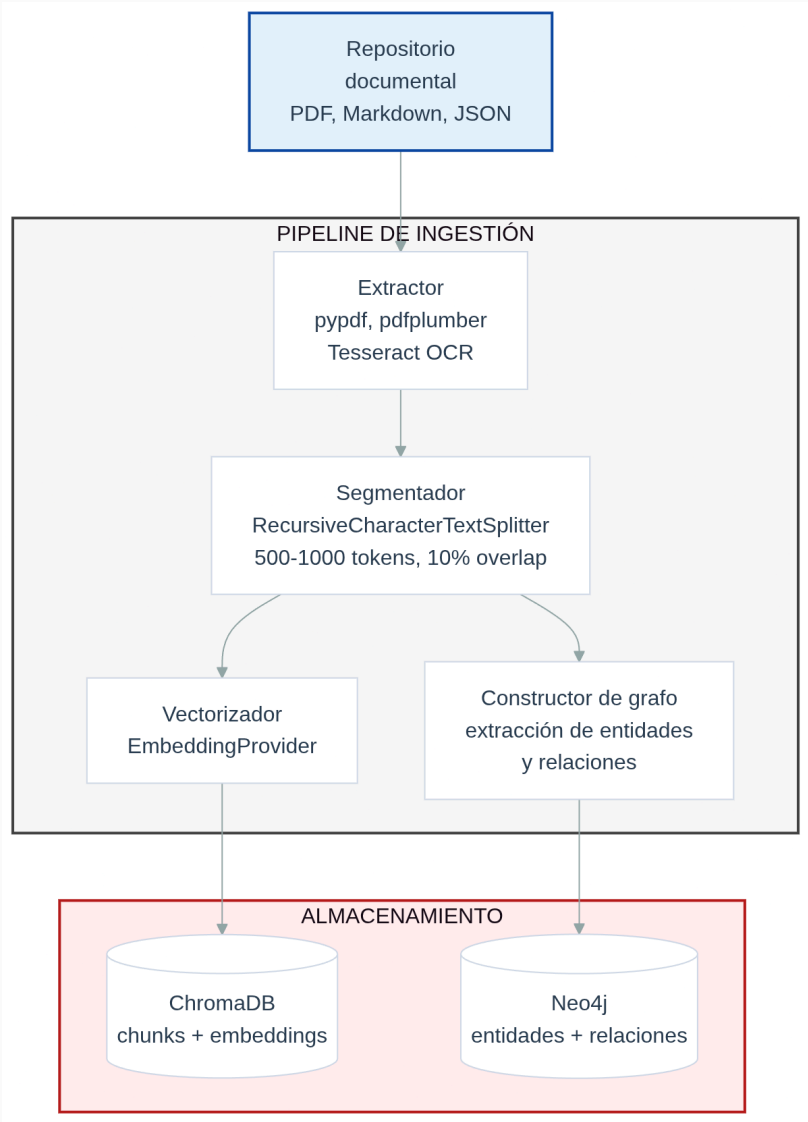
HuggingFaceEmbeddingAdapter. Consumirá modelos alojados en la Inference API de Hugging Face.

SentenceTransformersAdapter. Ejecutará modelos locales como all-MiniLM-L6-v2, adecuado cuando el costo o la latencia de un proveedor remoto resulten prohibitivos.

3.7 Pipeline de ingestión

El pipeline de ingestión operará como proceso batch independiente del endpoint de inferencia. En la Figura 8 se presenta el flujo del pipeline.

Figura 8. Pipeline de ingestión.



Se ejecutará bajo demanda o mediante tareas programadas. El flujo constará de cuatro etapas: extracción de texto, segmentación en chunks, bifurcación en vectorización (ChromaDB) y construcción de grafo (Neo4j), y registro de la versión de cada documento. La separación del pipeline respecto al endpoint garantizará que la actualización de la base de conocimiento no interrumpa el servicio.

3.8 Beneficios del diseño hexagonal

La adopción del patrón hexagonal entregará tres beneficios concretos. En primer lugar, la testabilidad del núcleo: las pruebas unitarias se ejecutarán con dobles de prueba (mocks o fakes) de los puertos, sin llamadas reales a proveedores de LLM ni a bases de datos. En segundo lugar, la independencia tecnológica: la migración de un proveedor se traducirá en un

nuevo adaptador, sin cambios en el núcleo. En tercer lugar, la coexistencia de estrategias: el equipo podrá alternar entre un modelo remoto (GPT-4o) para solicitudes de alta prioridad y un modelo local (Ollama con Llama 3.1) para las de baja prioridad, mediante un cambio de configuración.

3.9 Consideraciones de despliegue y operación

El servicio se empaquetará en una imagen Docker basada en python:3.11-slim. El despliegue se ejecutará en un clúster de Kubernetes, con un Ingress con terminación TLS y acceso restringido por token de servicio.

Para el monitoreo, expondrá métricas en formato Prometheus a través de /metrics: latencia por inferencia (histograma), llamadas al LLM (contador), costo acumulado en tokens (contador) y tasa de errores (contador). Cada inferencia se registrará con su trace_id en el sistema de logging centralizado.

La estrategia de caché se aplicará a dos niveles: las consultas a la base de conocimiento se cachearán por un tiempo corto (5 minutos) mediante Redis, y las llamadas al LLM no se cachearán por defecto dado que cada solicitud tendrá datos distintos. En escenarios de alto volumen, se contemplará caché semántica basada en similitud de embeddings.

Con la arquitectura ya definida, la siguiente sección presenta cómo se distribuyó el trabajo de documentación entre los tres integrantes del equipo, así como la asignación de proveedores de LLM que cada uno implementará en la próxima entrega.

4. Asignación de tareas del equipo

El desarrollo del presente documento se distribuye entre los tres miembros del equipo. En la Tabla 8 se presenta la distribución de responsabilidades.

Tabla 8. Asignación de tareas del equipo.

Responsable	Bloque	Alcance y secciones del documento
Elizabeth Toro Chalarca	Definición funcional y justificación de negocio	Secciones 1, 1.1, 1.2 y 1.3 (análisis comparativo). Describe el problema operativo, presenta la funcionalidad RAG con su justificación para el contexto agrícola colombiano, diseña el esquema explicativo de cuatro fases y construye la matriz de evaluación de funcionalidades candidatas. Su entregable responde a la pregunta: qué resuelve esta API y por qué se selecciona.
Santiago Andrés Cardona Julio	Ingeniería de datos y contrato de la API	Sección 2 completa (2.1, 2.2, 2.3). Identifica y clasifica las fuentes documentales, diseña el pipeline de ingestión, define los campos de entrada con sus reglas de validación, los campos de salida y los ejemplos de payload. Su entregable responde a la pregunta: qué datos entran, qué datos salen y qué conocimiento alimenta al sistema.
Edward Alejandro Rayo Cortés	Propuesta de arquitectura	Sección 3 completa (3.1 a 3.9). Diseña la arquitectura hexagonal con puertos y adaptadores, documenta los tres puertos del núcleo, describe los adaptadores intercambiables para LLM y para el repositorio de conocimiento, y define las consideraciones de despliegue, monitoreo y caché. Su entregable responde a la pregunta: cómo se construye y cómo se opera el servicio.

Adicionalmente, en la tabla 9 se muestra la asignación de LLMs que cada miembro implementará como parte de la propuesta del siguiente trabajo.

Tabla 9. Asignación de LLM

Responsable	Bloque	Justificación
Elizabeth Toro Chalarca	Gemini 1.5 Flash (Google)	Gemini [10,11] ofrece un tier gratuito con capacidad suficiente para procesar documentos PDF de forma directa, lo cual facilita la demostración de valor con las fuentes documentales del dominio agrícola.
Santiago Andrés Cardona Julio	GPT-4o mini (OpenAI)	GPT-4o [13,14] funciona muy bien combinando información recuperada (documentos, reglas, históricos) con generación en lenguaje natural, entregando respuestas útiles y contextualizadas.
Edward Alejandro Rayo Cortés	Claude (Anthropic)	Claude [9] se destaca por su capacidad de razonamiento sobre documentos heterogéneos y por su ventana de contexto de 200.000 tokens, lo cual lo hace adecuado para procesar en una sola inferencia el conjunto de fragmentos recuperados de la base de conocimiento.

Las conclusiones (sección 6) se redactarán de forma conjunta por los tres miembros, dado que cada persona aportará las conclusiones derivadas de su bloque de trabajo.

5. Conclusiones del caso de aplicación

A partir del diseño y la documentación de la API RAG para la selección inteligente de vehículo, se derivan las siguientes conclusiones principales.

1. La decisión de construir la funcionalidad como API agnóstica al consumidor, implementada en Python, garantiza la independencia tecnológica del servicio. Cualquier sistema autorizado se integrará mediante HTTP estándar, sin restricciones sobre su lenguaje o plataforma.
2. La adopción del patrón hexagonal (ports and adapters) separa el núcleo de dominio de la infraestructura externa. De esta manera, el cambio de proveedor del LLM entre OpenAI, Anthropic, Google, Ollama, Llama.cpp o Hugging Face se traduce en la implementación de un nuevo adaptador, sin modificaciones en la lógica de dominio. La misma ventaja aplica al repositorio de conocimiento, donde conviven adaptadores para base vectorial, base de grafos y un adaptador híbrido Graph RAG.
3. El contrato de API, formalizado con esquemas Pydantic y documentado vía OpenAPI, constituye el único punto de acoplamiento entre el servicio y sus consumidores. Cualquier cambio interno que respete el contrato no requerirá modificaciones en los sistemas cliente.
4. La base de conocimiento diseñada, con cinco categorías documentales especializadas en logística agrícola colombiana, constituye un activo reutilizable que habilitará futuros servicios RAG para otras funcionalidades del dominio agrícola.
5. La estrategia de Retrieval-Augmented Generation resulta adecuada para el contexto agrícola colombiano, donde las condiciones de transporte varían entre regiones y temporadas. La capacidad de actualizar la base de conocimiento sin modificaciones en el código mantendrá el servicio vigente ante cambios normativos, nuevas rutas o modificaciones en la flota vehicular.
6. El patrón de salida estructurada con trace_id garantiza que cada recomendación resulte accionable y auditable. El trace_id habilitará la trazabilidad completa de cada inferencia.

Consideraciones personales. El ejercicio de diseñar una funcionalidad RAG como API agnóstica al consumidor, apoyada en arquitectura hexagonal, permite comprender que el valor principal del diseño no reside en la tecnología de implementación, sino en la claridad del contrato expuesto y en la disciplina de abstraer las dependencias externas mediante puertos. Un contrato bien diseñado convierte al servicio en un activo consumible por cualquier sistema, presente o futuro, y los puertos desacoplan al núcleo de un ecosistema de proveedores de LLM y bases de conocimiento que evoluciona con rapidez.

6. Referencias

[1] FastAPI. (2025). FastAPI framework, high performance, easy to learn. Recuperado de https://fastapi.tiangolo.com/

[2] Chroma. (2025). ChromaDB. The AI-native open-source embedding database. Recuperado de https://www.trychroma.com/

[3] Pydantic. (2025). Pydantic. Data validation using Python type hints. Recuperado de https://docs.pydantic.dev/

[4] OpenAI. (2025). OpenAI Platform Documentation. Recuperado de https://platform.openai.com/docs

[5] Cockburn, A. (2005). Hexagonal architecture. Recuperado de https://alistair.cockburn.us/hexagonal-architecture/

[6] Neo4j. (2025). Neo4j Graph Database. Recuperado de https://neo4j.com/docs/

[7] Ollama. (2025). Get up and running with large language models locally. Recuperado de https://ollama.com/

[8] Hugging Face. (2025). The AI community building the future. Recuperado de https://huggingface.co/docs

[9] Anthropic. (2025). Claude API Documentation. Recuperado de https://docs.anthropic.com/

[10] Google. (2025). Gemini API Documentation. Recuperado de https://ai.google.dev/docs

[11] Google. (2025). Google Gen AI SDK for Python. Recuperado de https://github.com/google/genai-python

[12] LangChain. (2025). LangChain: Building applications with LLMs through composability. Recuperado de https://python.langchain.com/

[13] OpenAI. (2025). ChatGPT Documentation. Recuperado de https://platform.openai.com/docs/guides/chat

[14] OpenAI. (2025). GPT-4o Documentation. Recuperado de https://platform.openai.com/docs/models/gpt-4o

[15] Instituto Colombiano Agropecuario. (2025). Portal institucional ICA. Recuperado de https://www.ica.gov.co/

[16] Corporación Colombiana de Investigación Agropecuaria. (2025). AGROSAVIA - Corporación colombiana de investigación agropecuaria. Recuperado de https://www.agrosavia.co/

[17] Instituto Nacional de Vías. (2025). Portal INVÍAS. Recuperado de https://www.invias.gov.co/

[18] Instituto Nacional de Vigilancia de Medicamentos y Alimentos. (2025). Portal INVIMA. Recuperado de https://www.invima.gov.co/

[19] Ministerio de Transporte de Colombia. (2025). Portal del Ministerio de Transporte. Recuperado de https://mintransporte.gov.co/

[20] Departamento Administrativo Nacional de Estadística. (2025). Sistema de Información de Precios y Abastecimiento del Sector Agropecuario (SIPSA). Recuperado de https://www.dane.gov.co/index.php/estadisticas-por-tema/agropecuario/sistema-de-informacion-de-precios-sipsa

[21] Edge, D., Trinh, H., Cheng, N., Bradley, J., Chao, A., Mody, A., Truitt, S., Metropolitansky, D., Ness, R. O., & Larson, J. (2024). From Local to Global: A Graph RAG Approach to Query-Focused Summarization. arXiv:2404.16130. Recuperado de https://arxiv.org/abs/2404.16130

Apéndice A. Glosario de siglas y abreviaturas

A lo largo del documento se emplean diversas siglas y abreviaturas técnicas. En la Tabla A1 se presenta el glosario completo con el significado y la descripción de cada una, organizadas en orden alfabético.

Tabla A1. Glosario de siglas y abreviaturas.

Sigla	Significado	Descripción en el contexto del documento
ABC	Abstract Base Class	Clase base abstracta de Python. Se emplea para definir los puertos (interfaces) de la arquitectura hexagonal.
AGROSAVIA	Corporación Colombiana de Investigación Agropecuaria	Entidad pública colombiana que produce fichas técnicas de productos agrícolas consultadas como parte de la base de conocimiento.
API	Application Programming Interface	Interfaz de programación de aplicaciones. Se refiere tanto a la API REST del servicio RAG como a las APIs de los proveedores de LLM y de embeddings.
CLI	Command Line Interface	Interfaz de línea de comandos. Se emplea como adaptador de entrada del pipeline de ingestión.
FEFO	First Expired, First Out	Política de rotación de inventario donde el producto con la fecha de vencimiento más próxima se despacha primero. Se evaluó como funcionalidad candidata y se descartó por su inadecuación al enfoque RAG.
GGUF	GPT-Generated Unified Format	Formato de archivo para modelos de lenguaje cuantizados. El adaptador LlamaCppAdapter consumirá modelos en este formato.
GPT	Generative Pre-trained Transformer	Familia de modelos de lenguaje de OpenAI. El servicio soportará GPT-4o y GPT-4o-mini mediante el OpenAIAdapter.
HTTP	Hypertext Transfer Protocol	Protocolo de comunicación sobre el cual la API expondrá sus endpoints.
ICA	Instituto Colombiano Agropecuario	Entidad pública colombiana responsable de la sanidad agropecuaria. Sus documentos técnicos formarán parte de la base de conocimiento.
INVIAS	Instituto Nacional de Vías	Entidad colombiana responsable de la infraestructura vial. Los reportes sobre el estado de vías terciarias se incorporarán a la base de conocimiento.
INVIMA	Instituto Nacional de Vigilancia de Medicamentos y Alimentos	Entidad colombiana reguladora de la cadena de frío y el transporte de alimentos. Sus resoluciones se incluirán como documentos normativos en la base de conocimiento.
ISO	International Organization for Standardization	Organización internacional de normalización. Se referencia el formato ISO 8601 para la representación de fechas en el contrato de la API.

JSON	JavaScript Object Notation	Formato de intercambio de datos empleado tanto en el payload de entrada como en la respuesta del servicio.
LLM	Large Language Model	Modelo de lenguaje grande. Constituirá el componente de razonamiento del servicio RAG. El puerto LLMProvider abstraerá la comunicación con el proveedor del modelo.
Merma		Se refiere a la pérdida de producto que ocurre entre el punto de origen y la entrega al cliente final. Estas pérdidas tienen varias causas: deterioro por temperatura inadecuada durante el transporte, daño físico por empaque deficiente, vencimiento de la vida útil del producto antes de su venta, o manipulación incorrecta durante la carga y descarga.
ML	Machine Learning	Aprendizaje automático. En el análisis comparativo se evaluó la adecuación de cada funcionalidad al enfoque RAG frente a alternativas de ML como regresión, clasificación y optimización combinatoria.
MVP	Minimum Viable Product	Producto mínimo viable. Se emplea como criterio de evaluación en la matriz comparativa para valorar la complejidad de implementación.
OCR	Optical Character Recognition	Reconocimiento óptico de caracteres. El pipeline de ingestión empleará Tesseract OCR para extraer texto de documentos escaneados.
PDF	Portable Document Format	Formato de documento portátil. La mayoría de los documentos normativos y fichas técnicas de la base de conocimiento se encuentran en este formato.
RAG	Retrieval-Augmented Generation	Generación aumentada por recuperación. Patrón de arquitectura de IA donde un modelo de lenguaje recibe contexto documental recuperado en tiempo de inferencia para fundamentar sus respuestas.
REST	Representational State Transfer	Estilo arquitectónico para el diseño de APIs web. La API del servicio seguirá las convenciones REST con endpoints, métodos HTTP y códigos de estado estándar.
SIPSA	Sistema de Información de Precios y Abastecimiento del Sector Agropecuario	Sistema colombiano del DANE que publica boletines de precios por producto, plaza y calidad.
TLS	Transport Layer Security	Protocolo de seguridad para comunicaciones cifradas. La exposición del servicio en Kubernetes empleará un Ingress con terminación TLS.
UUID	Universally Unique Identifier	Identificador único universal. El campo trace_id de la respuesta de la API se generará como UUID v4 para garantizar la trazabilidad de cada inferencia.